

결 과 보 고 서

과제명: Ternary MAC Architecture with Scaling
for Energy-Efficient NPU Design

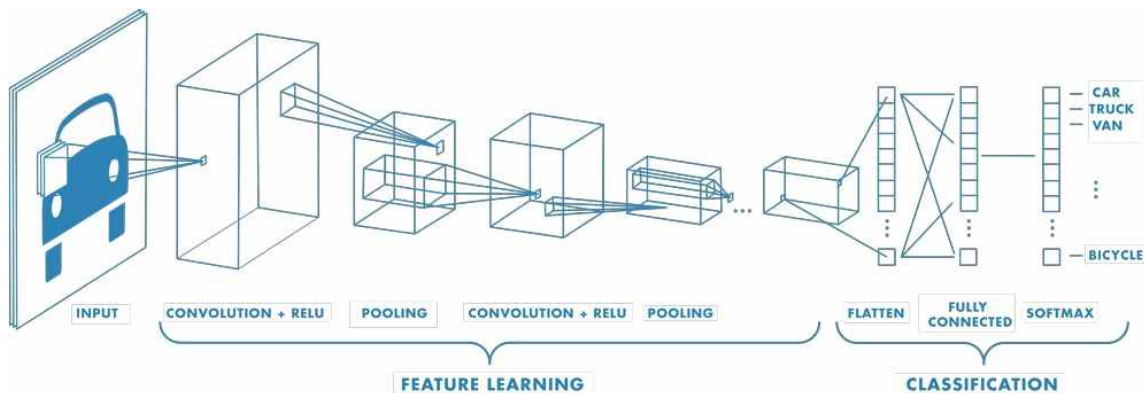
팀 명: TernaryX

경북대학교 IDEC 2024 창의 회로설계 챌린지

1. 과제 개요

최근 딥러닝 기술은 클라우드 기반 연산에서 벗어나 실시간/온디바이스 추론과 경량화 및 효율화 방향으로 발전하고 있다. 그중 합성곱 신경망(CNN)은 이미지, 영상, 센서맵 등 격자형 데이터 처리에 특화되어 있으며 국소 패턴을 계층적으로 추출하고 안정적인 일반화 성능을 보유해 여전히 산업 현장의 표준 모델로 사용되고 있다. 자율주행, 산업 비전, 의료 영상, 로봇틱스와 같은 응용 분야에서는 낮은 지연과 높은 신뢰성, 로컬 처리가 요구되므로 클라우드 의존 방식만으로는 한계가 있다. 이에 따라 엣지 단에서 CNN을 빠르고 효율적으로 실행할 수 있는 전용 하드웨어 가속기(CNN Accelerator)의 필요성이 커지고 있다.

본 과제는 NPU를 설계하여 입력-특징 추출-분류 흐름을 하드웨어 관점에서 재구성하고 연산과 메모리 이동의 균형을 고려한 데이터경로 중심 설계를 적용한다. 설계는 Verilog HDL을 기반으로 수행되며 Vivado 및 OpenROAD를 이용해 합성(Synthesis)과 물리 설계(Place & Route)를 진행한다. 성능 평가는 기능적 정확성, 처리속도, 전력 효율, 자원 사용률 등을 지표로 수행하며 테스트벤치와 로그를 통해 결과의 재현성과 신뢰성을 확보한다. 최종적으로 정확도, 응답시간, 에너지 효율을 동시에 만족하는 NPU를 구현함으로써 엣지 환경에서의 실시간 인공지능 추론을 가능하게 하고 향후 자율주행, 로봇틱스, 산업 비전 분야에 활용 가능하도록 한다.



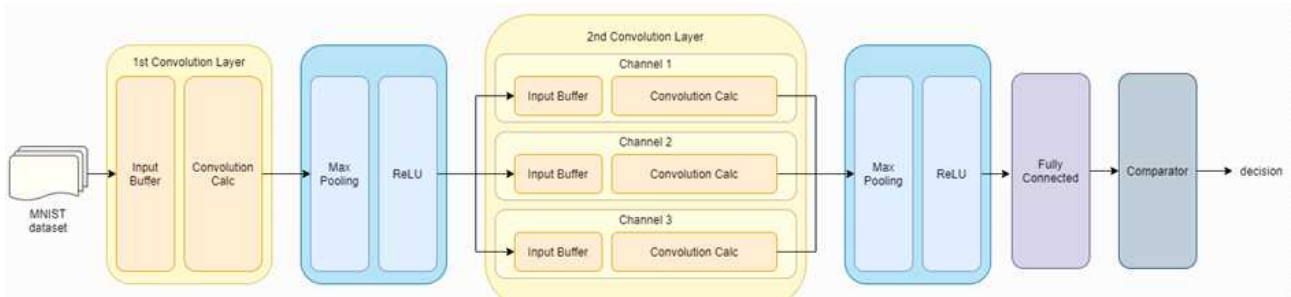
[그림 1] Convolution Neural Network(CNN) architecture

2. 설계 목표

본 설계의 목표는 Ternary MAC 구조와 Adder Tree Pipelining 기법을 적용하여 CNN 가속기의 연산 효율과 타이밍 안정성을 동시에 확보하는 것이다. 연산 경로를 단순화하면서도 정확도를 유지할 수 있는 구조를 구현하고 CNN 연산의 전력 소모를 최소화하여 에너지 효율(TOPS/W)이 높은 NPU 설계를 주된 목표로 한다.

또한, 합성곱 계층의 연산 병목을 완화하기 위해 데이터 흐름 중심의 병렬 구조를 설계하고 파이프라인 단계 간 균형을 최적화하여 지연과 자원 사용률 사이의 균형을 확보하는 것을 목표로 한다. 이를 통해 타이밍 제약 내에서 안정적인 처리 속도를 유지하고 Worst Slack ≥ 0 ns 수준의 합성 타이밍 수렴을 달성할 수 있는 구조를 지향한다.

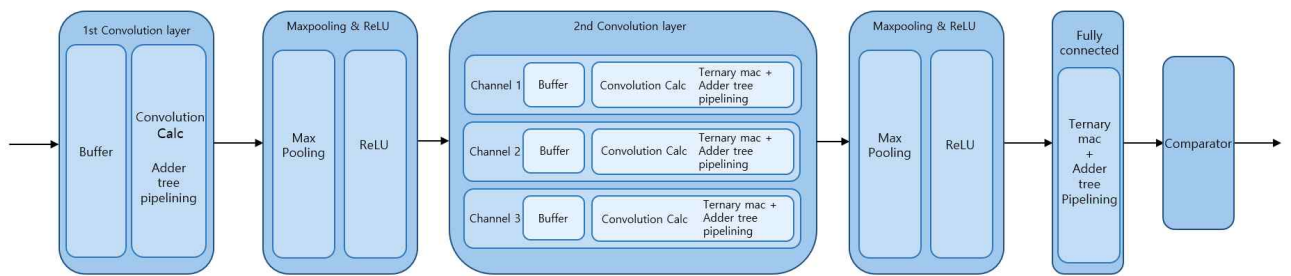
마지막으로, Fully Connected(FC) 및 Comparator 단계는 전체 시스템의 분류 정확도와 응답속도에 직접적인 영향을 미치므로 전단부에서 학습된 특징 정보를 손실 없이 전달할 수 있는 경량 구조로 설계하는 것을 목표로 한다. 이를 통해 Ternary 기반 CNN 가속기 전체가 정확도, 전력, 면적 측면에서 균형을 이루는 효율적인 NPU 구조를 구현하는 것을 최종 목표로 한다.



[그림 2] Reference code Top module Architecture

3. 설계 및 시뮬레이션

1. 설계 아키텍처



[그림 3] proposed CNN accelerator Top module architecture

본 NPU의 전체 아키텍처는 입력 스트림을 순차적으로 1차 합성곱층(Conv1) → MaxPooling & ReLU → 2차 합성곱층(Conv2) → MaxPooling & ReLU → 완전연결층(FC) → 비교기(Comparator) 순서로 처리하는 구조로 되어 있다. 각 단계는 valid_out_1부터 valid_out_6까지의 유효 신호(valid pipeline)를 통해 시간적으로 정렬되어 데이터 흐름이 일정하게 유지된다.

1차 합성곱층은 라인 버퍼와 합산 트리를 이용한 Adder Tree Pipelining 구조로 구성되어 있으며, 5×5 커널을 사용해 입력 특징을 추출한다. 이후 2×2 MaxPooling과 ReLU를 적용해 비선형성을 부여하고 데이터 크기를 줄인다. 2차 합성곱층에서는 Adder Tree Pipelining 구조와 곱셈기를 제거한 Ternary MAC (−1/0/+1) 구조를 사용하여 연산 효율을 높였으며, 다시 한 번 풀링과 ReLU를 거쳐 특징 맵을 압축한다.

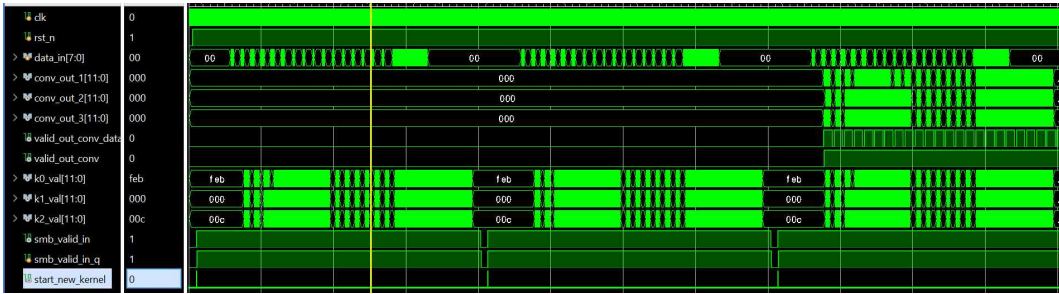
마지막으로 Fully Connected 계층은 3채널의 풀링 결과를 입력받아 시분할 방식으로 처리하며 Ternary 연산과 6단계 Adder Tree 파이프라인을 통해 연산을 수행한다. 마지막 Comparator 단계에서는 10개의 클래스 점수 중 최대값과 해당 인덱스를 선택하여 최종 판단 결과를 출력한다. 전체 데이터 경로는 12 비트 정수 포맷으로 통일되어 있으며, 각 모듈 간의 valid 신호 체인을 통해 정확한 파이프라인 동작이 보장된다.

2. 최적화 방법

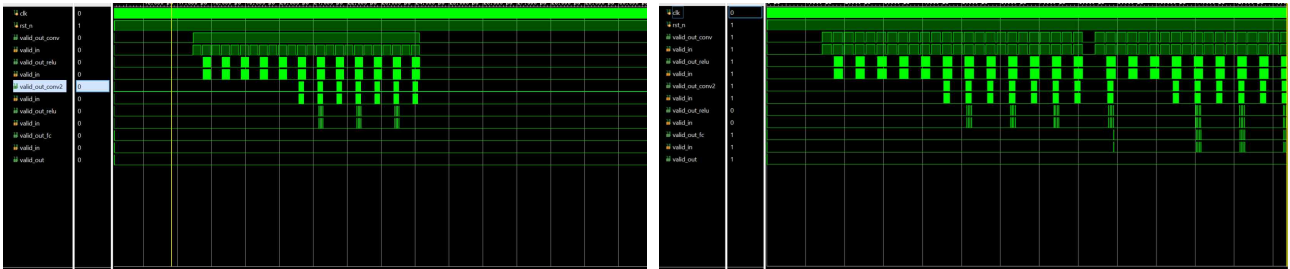
2-1. Systolic Array 설계 및 한계점

초기 해결 방안으로, 범용 NPU에 널리 사용되는 Systolic Array(SA) 구조를 합성곱 계층에 도입하고자 하였다. 이를 위해 PE(Processing Element), SA, 버퍼(SMB, WB), MainController 등 총 5개의 모듈을 설

계했으며, 설계하는데 성공한 첫 번째 합성곱 계층에 대한 단위 모듈 시뮬레이션 결과는 아래와 같다.



[그림 4] Conv1_layer Systolic Array로 구현한 연산 파형

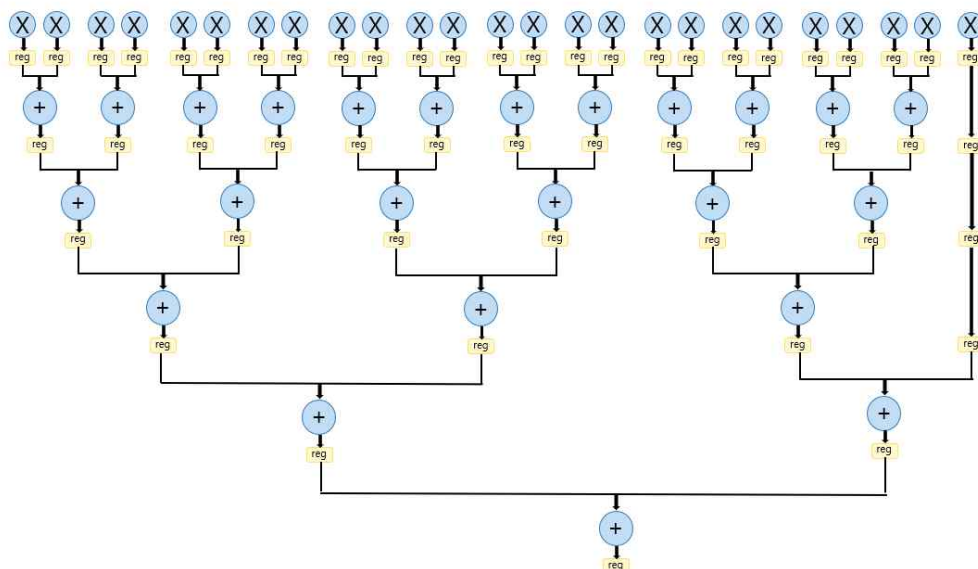


[그림 5] 모든 모듈의 제어 신호 전달 파형

SA 구조를 CNN 코드에 통합하고 시뮬레이션을 수행한 결과, 최종 decision 값이 출력되지 않는 문제가 발생했다. 이는 모듈 간 제어 신호가 서로 맞지 않아 생긴 문제로 판단되었다. 근본적인 원인은 초기 설계와 SA 구조의 데이터 스트리밍 불일치였다. SA는 데이터가 끊임없이 스트리밍 방식으로 들어와야 하고 핸드셰이크 방식으로 통신해야 하지만, 기존 코드는 데이터를 배치 단위로 한 번에 내보내는 단방향 푸시 방식이었다. 이렇게 데이터와 제어 신호를 다루는 방식이 달라 모듈간의 흐름이 끊기는 것을 볼 수 있었다.

이 문제를 해결하기 위해서는 전체적인 수정이 필요하기에 SA 적용은 추후 과제로 넘기고, 현 과제에서는 다른 대안을 통해 최적화를 진행했다.

2-2. Adder Tree Pipelining



[그림 6] Adder tree pipelining 구조

대안으로 채택된 Adder Tree(AT)는 각 덧셈 단계 사이에 레지스터를 삽입하기에 적합한 구조로, 레지스터 증가에 따른 면적 상승이라는 트레이드오프가 있으나 긴 연산 경로를 다수의 클록 사이클로 분할하

여 최대 동작 주파수(Fmax)와 타이밍 여유를 향상시킨다. 또한 규격화된 트리 토폴로지와 짧아진 배선 구간은 라우팅 효율을 높여 동적 전력 소모를 줄이고, 배치·배선 과정의 예측 가능성을 개선한다.

본 설계는 5×5 커널에서 생성된 25개 곱셈 결과를 균형형 파이프라인드 AT로 25→13→7→4→2→1 단계로 축약한다. 각 단계는 두 항을 가산하고 홀수로 남는 항은 다음 단계로 전달(pass-up)하여 균형을 유지하며, 단계 사이 레지스터로 스테이지당 가산기 1개만 통과하도록 제한한다. 파이프라인 지연은 $LAT_TOT = MUL_LAT + 5 + POST$ 로 정의하고, 출력 데이터와 동기화를 위해 valid 신호를 동일 길이로 지연시켰다. 다채널(conv2) 환경에서는 채널별 파이프라인 깊이를 일치시키고 valid를 동기화하여 동일 좌표의 결과가 한 사이클에 정렬되도록 하였다. 비트폭은 입력폭(IN_W)에 $\lceil \log_2 25 \rceil$ 를 더한 $SUM_W = IN_W + 5$ 로 설정하여 오버플로를 방지하면서 가산기 폭과 캐리 체인을 최소화하였다(모든 연산은 signed 일관성 유지).

시뮬레이션 결과, 임펄스 입력으로 측정한 지연이 계산된 LAT_TOT와 일치하였고, 난수 벡터에 대해 소프트웨어 기준 모델(동일 라운딩·쉬프트·포화 규칙)과 RTL 출력이 사이클 단위로 일치하였다. 포화 경계 및 부호 혼합 입력에서도 기대 동작이 확인되었다. 종합하면, 균형형 트리-단계별 파이프라인-최소 필요 비트폭이라는 원칙을 통해 고클록 타이밍 안정성과 면적·전력 효율을 동시에 달성하였으며, 유효 신호 정렬 규약으로 상위 모듈 통합의 신뢰성 또한 확보하였다.

2-3. Ternary Mac

1) 이론적 배경

Ternary MAC은 가중치를 실수 대신 $\{-1, 0, +1\}$ 의 세 가지 값으로 제한하여 곱셈기를 제거하는 경량 신경망 연산 구조이다. 일반적인 곱셈-누산(MAC) 연산인 $z = x * w$ 를 근사식 $z \approx \alpha * (x \oplus \hat{w})$ 로 표현할 수 있으며, 이때 w 는 삼진화된 가중치, α 는 채널별 스케일 팩터이다.

즉, 입력 x 에 대해 “더하기(+1) / 빼기(-1) / 생략(0)”의 세 가지 연산만 수행하므로, 곱셈 연산이 단순한 선택 연산으로 치환된다.

$$\begin{cases} z = x * w \approx x * (\alpha \hat{w}) = \alpha * (x \oplus \hat{w}) \\ x^{next} = g(z) \end{cases}$$

가중치의 절대값이 임계값 Δ 보다 크면 부호(+/-)만 남기고, 그 이하일 경우 0으로 만든다. 즉, 곱셈기를 제거하고, 0 값의 가중치에 대해 연산을 생략할 수 있게 하여 전력과 연산량을 줄인다.

$$\hat{w}_i = f(w_i | \Delta) = \begin{cases} +1 & \text{if } w_i > \Delta \\ 0 & \text{if } |w_i| \leq \Delta \\ -1 & \text{if } w_i < -\Delta \end{cases}$$

임계값 Δ 는 가중치의 평균 절대값 $E(|w|)$ 에 상수 계수를 곱해 추정한다. 이렇게 계산된 Δ 를 기준으로 0과 ± 1 을 구분하면, 가중치의 중요도에 따라 연산 효율성을 자연스럽게 확보할 수 있다.

$$\Delta^* \approx 0.75 E(|w|) \approx \frac{0.75}{n} \sum_{i=1}^n |w_i|$$

삼진화로 인해 손실된 표현력은 스케일 팩터 α 를 통해 복구된다. α 는 Δ 를 넘는 가중치들(즉, 0이 아닌 항)의 평균 절대값으로 정의된다.

결과적으로 Ternary MAC의 연산 절차는 (1) Δ 계산 → (2) $\{-1, 0, +1\}$ 투사 → (3) α 추정 → (4) 부호 선

$$\alpha_{\Delta}^* = \frac{1}{|I_{\Delta}|} \sum_{i \in I_{\Delta}} |w_i|$$

택 연산 수행으로 구성된다. 이 구조는 곱셈기 없이 부호 선택과 가산기만으로 동작하며, 채널별 α 를 통해 정확도를 유지하면서도 연산 및 전력 소모를 크게 절감할 수 있다.

2) 학습 및 가중치 추출 python code

파이썬 학습 및 내보내기 파이프라인은 위의 삼진화 이론을 그대로 구현하였다. 먼저 TernaryQuantFn 모듈이 가중치 w 를 임계값 Δ 기준으로 $\{-1, 0, +1\}$ 로 투사하며, Δ 는 채널별 평균 절대값 $E(|w|)$ 에 사용자 정의 계수 $t(=0.7)$ 를 곱하여 계산된다. 0이 아닌 가중치만을 이용해 α 를 추정하며, 이는 평균 절대값을 기반으로 한다. 이 과정을 통해 얻은 $(w^{\wedge}, \alpha)$ 쌍은 실수 연산 대신 $+x / -x / 0$ 선택 누적 후 α 스케일 적용 방식으로 사용된다. 즉, 실제 연산은 덧셈, 부호 반전, 제로 스킵으로만 구성된다.

학습 중에는 정수 경로 일치를 위해 FakeQuantQ2_6STE와 ArithmeticRightShiftSTE를 사용하여, α , bias의 Q2.6 정수 라운딩 및 시프트를 시뮬레이션한다. 역전파는 Straight-Through Estimator(STE)를 사용해 미분 불가능한 구간을 통과시킴으로써 학습이 가능하도록 처리된다. 삼진화는 conv2와 FC 계층에만 적용되며 conv1은 INT8 기반의 Q1.15 스케일 정규화 방식을 사용한다. TernaryConv2d는 입력 채널별 5x5 합을 구한 뒤 22비트 누적 범위 내에서 클램핑하고, 스케일 $\alpha(Q2.6)$ 를 곱한 후 라운딩 시프트로 정수화한다. TernaryLinear는 48탭 누적을 28비트 범위로 제한하여 동일한 스케일 및 바이어스 처리를 수행한다. 최종적으로 export_txt_from_model() 함수가 conv2와 FC의 $w^{\wedge}$, α , bias를 각각 HEX 파일(Q2.6 형식)로 내보낸다. 이 파일들은 RTL 설계에서 \$readmemh로 불러와 그대로 사용되며, PyTorch 학습 모델과 하드웨어의 결과가 수치적으로 일치하도록 보장한다.

```
# ===== Ternary (QAT with STE) =====
class TernaryQuantFn(torch.autograd.Function):
    @staticmethod
    def forward(ctx, w, delta):
        pos = (w > delta).to(w.dtype)
        neg = (w < -delta).to(w.dtype)
        return pos - neg # {-1,0,+1}
    @staticmethod
    def backward(ctx, g):
        return g, None
```

[그림 7] TernaryQuantFn 클래스 - 가중치 삼진화 연산 구현

```
def ternary_with_alpha(w, t=TERNARY_T, per_channel_dim=0):
    dims = list(range(w.dim()))
    red = dims[:]; red.pop(per_channel_dim)
    mean_abs = w.abs().mean(dim=red, keepdim=True)
    delta = t * mean_abs
    tern = TernaryQuantFn.apply(w, delta)
    mask = tern.ne(0).to(w.dtype)
    denom = mask.sum(dim=red, keepdim=True).clamp_min(1.0)
    alpha = (w.abs()*mask).sum(dim=red, keepdim=True)/denom
    return tern, alpha
```

[그림 8] ternary_with_alpha() 함수 - 임계값 Δ 계산 및 스케일 α 추정

[그림 9] TernaryConv2d 클래스 - 합성곱 계층의 삼진 MAC 연산 구현

```

class TernaryConv2d(nn.Module):
    # conv2_calc.v (SUM_W = 22 bits)
    def __init__(self, in_ch, out_ch, k=5, stride=1, padding=0, t=TERNARY_T, bias=True):
        super().__init__()
        self.weight_fp = nn.Parameter(torch.empty(out_ch, in_ch, k, k))
        self.bias = nn.Parameter(torch.zeros(out_ch)) if bias else None
        self.stride, self.padding, self.t = stride, padding, t
        nn.init.kaiming_normal_(self.weight_fp, nonlinearity='relu')
        self.ACC_MAX = (1 << (CONV2_ACCUM_BITS - 1)) - 1
        self.ACC_MIN = -(1 << (CONV2_ACCUM_BITS - 1))

    def forward(self, x):
        # x shape: (N, C_in=3, H_in, W_in)
        tern_w, alpha_f = ternary_with_alpha(self.weight_fp, t=self.t, per_channel_dim=0)
        # tern_w shape: (C_out=3, C_in=3, 5, 5)

        # --- [FIX] Emulate RTL's 3 separate Adder Trees (AT1, AT2, AT3) ---
        psum_f32 = 0.0
        for ic in range(x.size(1)): # C_in=3
            # 1. 1-channel convolution (emulates one Adder Tree)
            # x[:, ic:ic+1] -> (N, 1, H, W)
            # tern_w[:, ic:ic+1] -> (C_out, 1, 5, 5)
            # groups=C_out (Depthwise) ensures C_out filters apply to C_in=1
            # We must simulate C_out convolutions, each on 1 input channel

            # This is complex. Let's simplify:
            # F.conv2d(x_in, w_in) does sum(x_ic * w_oc_ic)
            # We need sum( clamp(x1*w1, 22b) + clamp(x2*w2, 22b) + clamp(x3*w3, 22b) )

            # F.conv2d(x, tern_w, ...) already calculates sum(x_ic * w_oc_ic)
            # What RTL does is sum_ic( AdderTree(x_ic * w_oc_ic) )
            # The AdderTree(x*w) clamps the sum of 25 products to 22 bits.

            # Let's emulate sum1, sum2, sum3
            # x_ic: (N, 1, H_in, W_in)
            x_ic = x[:, ic:ic+1, ...]

            # w_oc_ic: (C_out, 1, 5, 5)
            w_oc_ic = tern_w[:, ic:ic+1, ...].to(x.dtype)

            # sum_ic = F.conv2d(x_ic, w_oc_ic)
            # This isn't right. F.conv2d(N,1,H,W) with (C_out,1,K,K) is what we need.

            sum_ic_f32 = F.conv2d(x_ic, w_oc_ic, bias=None, stride=self.stride, padding=self.padding)

            # 2. [FIX] Clamp each input channel's sum to 22-bits (emulating sum1, sum2, sum3)
            sum_ic_clamped = ClampSTE.apply(sum_ic_f32, self.ACC_MIN, self.ACC_MAX)

            # 3. Add to total psum_raw
            psum_f32 = psum_f32 + sum_ic_clamped

class TernaryLinear(nn.Module):
    # fc.v: sl (PROD_W+5:0) = 28 bits
    def __init__(self, in_f, out_f, t=TERNARY_T, bias=True):
        super().__init__()
        self.weight_fp = nn.Parameter(torch.empty(out_f, in_f))
        self.bias = nn.Parameter(torch.zeros(out_f)) if bias else None
        self.t = t
        nn.init.kaiming_uniform_(self.weight_fp, a=np.sqrt(5))
        self.ACC_MAX = (1 << (FC_ACCUM_BITS - 1)) - 1
        self.ACC_MIN = -(1 << (FC_ACCUM_BITS - 1))

    def forward(self, x):
        tern_w, alpha_f = ternary_with_alpha(self.weight_fp, t=self.t, per_channel_dim=0)
        y_acc_f32 = F.linear(x, tern_w.to(x.dtype), bias=None)
        y_acc = ClampSTE.apply(y_acc_f32, self.ACC_MIN, self.ACC_MAX)
        alpha_q = FakeQuantQ2_6STE.apply(alpha_f[:,0]).view(1, -1)
        bias_q = FakeQuantQ2_6STE.apply(self.bias).view(1, -1)
        s = ArithmeticRightShiftSTE.apply(y_acc * alpha_q, SHIFT_TERNARY, ROUND_TERNARY)
        b = ArithmeticRightShiftSTE.apply(bias_q, SHIFT_TERNARY, ROUND_TERNARY)
        y = torch.clamp(s + b, SAT_MIN, SAT_MAX)
        return y

```

[그림 10] TernaryLinear 클래스 – 완전연결 계층의 삼진 연산 구현

3) 칩 내부에서의 ternary 적용 방법

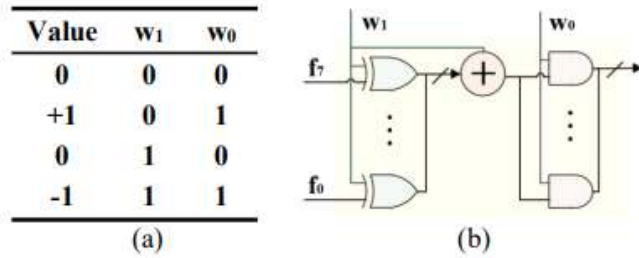
칩 내부에서는 전체 데이터 경로가 conv1(INT8 QAT) → conv2(Ternary) → FC(Ternary) → Comparator 순으로 구성되어 있다. conv1은 일반 정수 합성곱으로 작동하며, conv2와 FC에서만 Ternary MAC이 적용된다. 각 탭의 삼진 가중치 w^h 는 두 비트로 부호화되어 입력 신호에 대해 *****부호 선택 + 가산*****으로 동작한다. 즉,

$w^h=+1$: 입력을 그대로 더함

$w^h=-1$: 입력 부호를 반전시켜 더함

$w^h=0$: 해당 입력을 건너뛴 (제로 스킵)

이 구조는 [그림 11]의 표 및 회로도로 표현할 수 있으며, 곱셈기 없이 MUX와 가산기 트리만으로 구현된다.



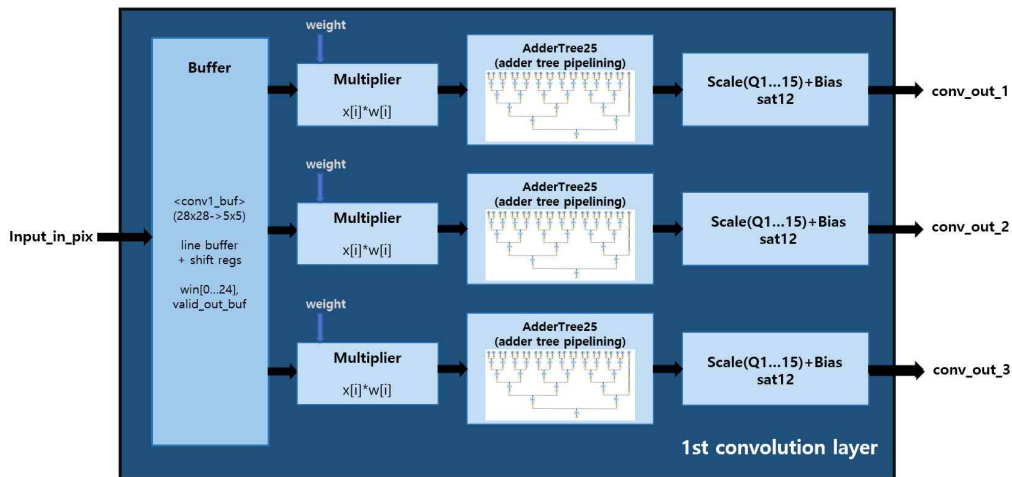
[그림 11] Ternary MAC 구현

Conv2 단계에서는 3채널 입력(12×12)을 받아 5×5 윈도우 단위로 슬라이딩하며, 채널별 Adder Tree(22비트 누적 폭)에서 부분합을 계산한다. 그 후 각 채널의 α (Q2.6)와 bias를 더하고 12비트로 포화시켜 출력한다. FC 단계에서는 3×4×4=48개의 입력 벡터에 대해 48탭 Ternary MAC을 수행하고, α 와 bias 적용 후 12비트 정수로 출력된다. 마지막 Comparator 모듈은 10개 클래스 점수 중 최대값을 선택한다. 모든 모듈은 valid 파이프라인을 통해 정렬되어, conv2와 FC의 누적 폭(22b, 28b), Q2.6 라운딩, 12b 포화 규칙이 Python QAT 모델과 완전히 일치하도록 설계되었다.

따라서 소프트웨어 학습 결과와 RTL 연산 결과가 1:1로 정합되며, 0 가중치 비율이 높을수록 Zero-skip 효과로 전력 효율이 더욱 향상된다. 결국, 칩 내부의 Ternary MAC 구조는 곱셈 없는 부호 선택 가산 방식, 채널별 α 스케일의 정수화 처리, 그리고 정확히 규정된 누적 폭과 포화 연산의 조합으로 구성되어 있다. 이를 통해 연산 면적과 전력 소모를 줄이면서도 학습 모델과의 수치적 정합성을 유지할 수 있다.

3. 모듈별(레이어별) 최적화 적용 방법

3-1. convolution 1



[그림 12] convolution 1 layer 내부 구조

Conv1 레이어는 28×28 크기의 8비트 영상을 입력받아, 내부 버퍼(conv1_buf)를 통해 5×5 윈도우 단위로 스트리밍 처리한다. 이 버퍼는 시프트 레지스터와 라인 FIFO가 결합된 구조로 한 픽셀 입력마다 5×5 윈도우가 자동 갱신되고 유효 신호(valid)가 함께 발생한다. 생성된 윈도우는 3개의 독립 채널로 전달되며, 각 채널은 정수 곱-누산 → 가산트리 파이프라이닝 → 스케일/바이어스 적용 → 12비트 포화 순으로 연산된다.

합성곱 연산은 AdderTree pipelining 구조를 사용해 25개의 곱셈 결과를 단계적으로 더하며 중간에 파이프라인 레지스터를 삽입해 연산 지연을 균등하게 나눴다. 이로써 곱셈기의 팬아웃과 결합 지연이 줄어들고, 회로의 최대 동작 주파수(Fmax)가 상승했다. 결과적으로 동일한 클록에서도 안정적인 타이밍 마

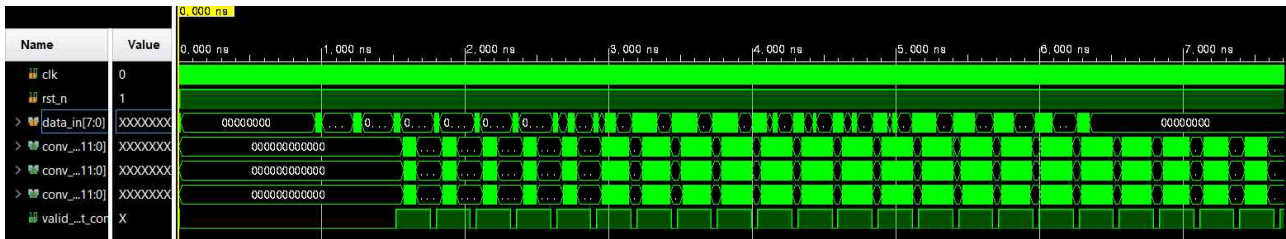
진을 확보할 수 있었다.

양자화 과정에서는 INT8 가중치, 채널별 스케일(Q1.15), INT8 바이어스를 사용한다. 하드웨어에서는 누산 결과에 스케일을 곱하고, 반올림 후 15비트 산술 우시프트(>>15)를 적용해 12비트로 포화시킨다.

이 방식은 정밀도 손실을 최소화하고, 누적기의 비트 성장을 억제하며, 다음 단계(conv2, maxpooling)와 12비트 데이터 일관성을 유지한다.

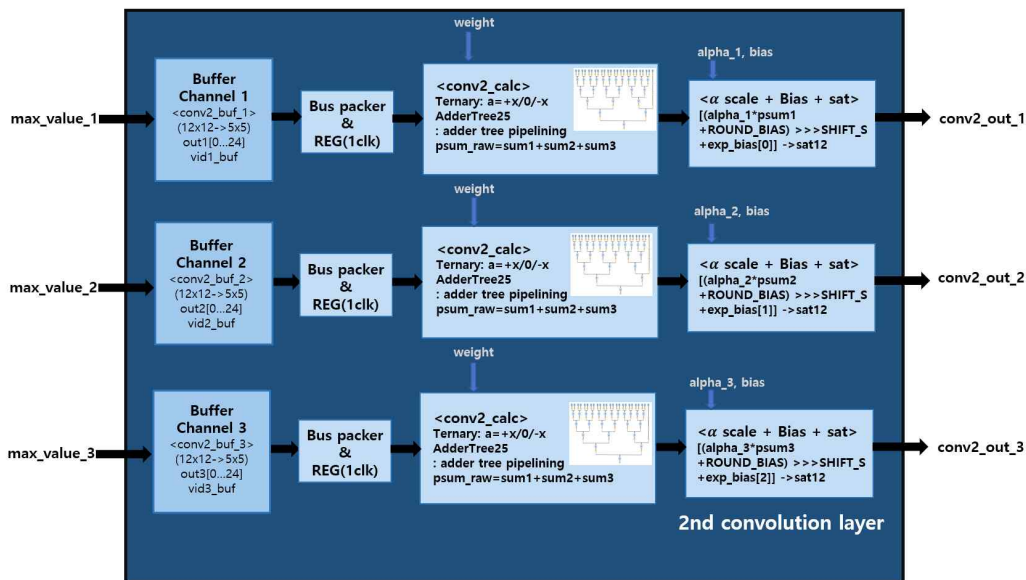
전체 연산 경로는 Multiplier → AdderTree25(pipelined) → Scale(Q1.15)+Bias → sat12로 구성된다. 모든 출력 채널은 동일한 파이프라인 깊이로 정렬되어, conv_out_1, conv_out_2, conv_out_3가 동시에 유효해진다.

검증은 PyTorch QAT 모델을 기준으로 RTL 시뮬레이션 결과(conv1_out_ch1~3, 24×24 맵)를 비교했으며 절대 오차 0을 확인했다. 또한 파형 분석을 통해 파이프라인 삽입 후에도 valid 신호가 정상적으로 정렬됨을 검증하였다. 결과적으로 Conv1 레이어는 정확도를 유지하면서도 연산 속도와 타이밍 안정성을 모두 향상시켰다.



[그림 13] conv1 simulation 파형

3-2. convolution 2



[그림 14] convolution 2 layer 내부 구조

Conv2 레이어는 pool1 단계에서 생성된 12×12 크기의 12비트 3채널 특성맵을 입력으로 받아 처리한다. 입력 데이터는 채널 분리형 버퍼(conv2_buf)를 통해 각 채널별로 5×5 윈도를 생성하고, 총 8×8개의 위치(64패치)를 스트리밍 방식으로 출력한다. 이 과정에서 버스 패커가 세 채널의 25탭 데이터를 한 사이클에 병합하여 후단으로 전달하며, 내부 레지스터(1clk)를 이용해 파이프라인 지연을 균등하게 맞춘다. 전체 구조는 세 개의 출력 채널이 병렬로 구성되어 있으며, 각각 독립적으로 conv2_out_1, conv2_out_2, conv2_out_3을 생성한다.

Conv2의 핵심 연산은 Ternary MAC 구조이다. 가중치는 {-1, 0, +1} 값으로 제한되어 있어 곱셈기가 필요 없으며, 각 탭에서는 입력 데이터에 대해 +x, 0, -x 중 하나를 선택해 단순한 부호 가산만 수행한다. 채널별 AdderTree25에서 부분합(sum1~sum3)을 계산한 뒤, 이를 합산해 psum_raw를 만든다. 모든 가산트리는 중간 단계마다 레지스터를 삽입한 adder tree 파이프라이닝 구조로 구현되어, 연산 지

연을 균등하게 분할한다. 이 방식은 곱셈기 제거로 인한 면적 및 전력 절감 효과는 물론, 누산 경로의 팬인.팬아웃을 줄여 Fmax 향상과 타이밍 안정성 확보에도 기여한다.

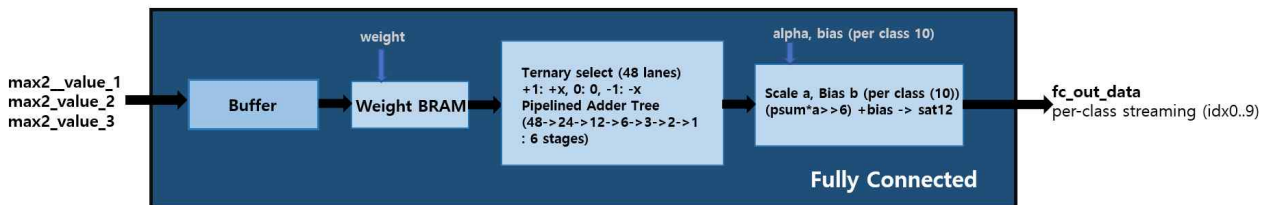
정규화 단계에서는 출력 채널별 스케일 $\alpha(Q2.6)$ 과 바이어스 $b(Q2.6)$ 를 적용한다. 이 스케일과 바이어스는 PyTorch QAT에서 생성된 값과 1:1로 대응하며, RTL에서도 동일한 라운딩 규칙을 사용하여 소프트웨어와 하드웨어의 결과 정합성을 유지한다. 또한 입력 유효 신호(valid)는 버퍼, 패커, 가산트리, 정규화 블록 전반에 동일한 파이프라인 깊이로 전달되어 세 출력 채널의 타이밍이 완벽히 일치한다.

검증 과정에서는 (1) PyTorch QAT 환경에서 기준 맵(8×8)을 생성하고, (2) RTL 시뮬레이션의 conv2_out 결과와 픽셀 단위로 비교하여 완전 일치를 확인했으며 (3) 96사이클 스트림의 타이밍이 u_pool2 입력 규격과 정확히 일치함을 파형 분석으로 검증했다. 결과적으로 Conv2는 삼진화 기반의 곱셈기 제거, 가산트리 파이프라이닝에 의한 연산 속도 향상, 그리고 채널별 정규화를 통한 정확도 보전을 동시에 달성하였다.



[그림 15] conv2 simulation 파형

3-3. fully connected



[그림 16] Fully connected layer 내부 구조

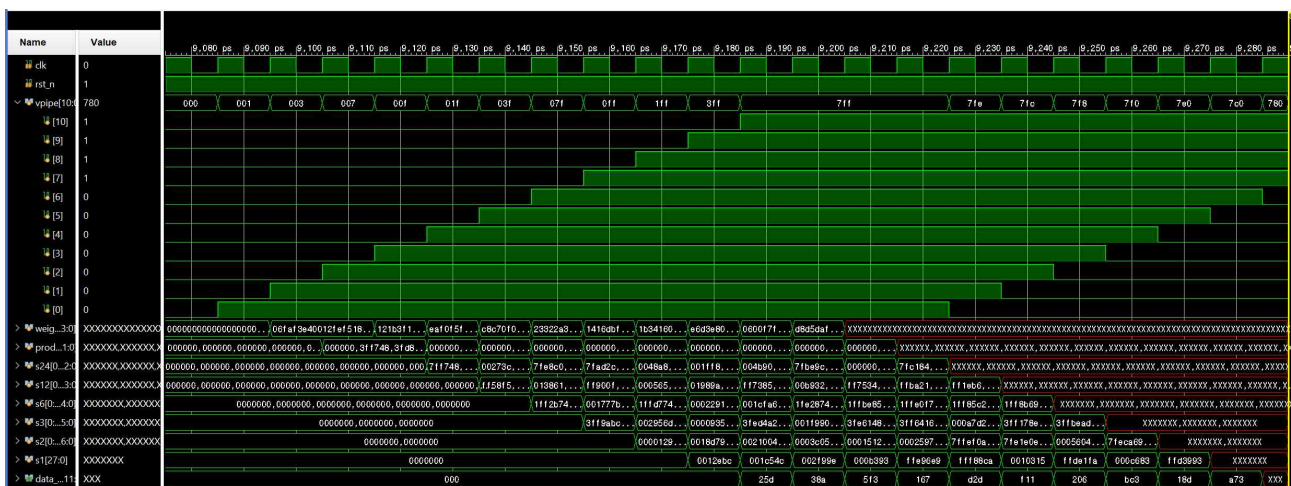
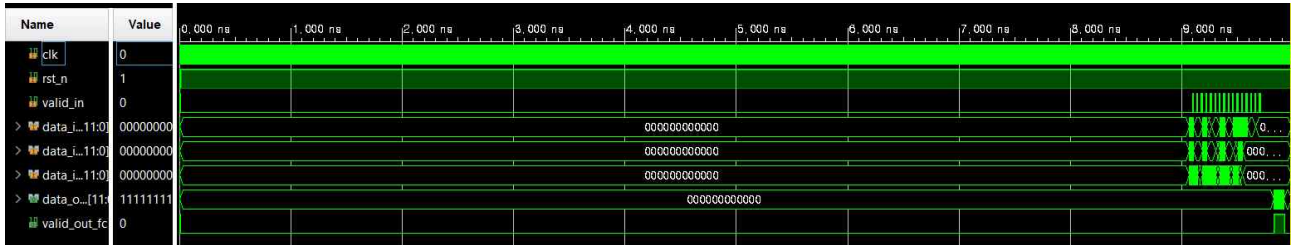
첨부 도식과 같이 FC는 입력 버퍼 → Weight BRAM → Ternary 선택기(48 lanes) → 파이프라인드 가산트리 → 스케일/바이어스 정규화 → 12b 포화로 구성된다. 입력은 pool2의 4×4×3 맵을 CHW 순서 (16,16,16 → 총 48값)로 버퍼링하여 한 번에 48개 탭을 병렬 공급한다. 가중치는 \$readmemh로 적재되는 fc_weight.txt(−1/0/+1)와 채널별 fc_scale.txt, fc_bias.txt(Q2.6)로 분리 보관하며 BRAM에서 클래스별 (0~9)로 스트리밍된다.

핵심은 Ternary MAC으로 각 탭에서 곱셈기를 없애고 $w \in \{+1, 0, -1\}$ 에 따라 $+x / 0 / -x$ 를 선택만 한다. 이렇게 얻은 48개 신호는 파이프라인드 가산트리(48→24→12→6→3→2→1, 총 6단)로 누산되어 psum_raw를 만든다. 각 층 사이에 레지스터를 삽입해 지연을 균등화함으로써 Fmax 향상과 타이밍 여유를 확보했고 곱셈기 제거로 면적/전력도 동시에 절감했다. 누산기는 사양에 맞춰 28비트로 잡아 오버플로를 방지했다.

정규화는 학습 단계에서 추정해 둔 클래스별 $\alpha(Q2.6)$, $b(Q2.6)$ 를 적용한다. 구현은 RTL 규칙에 맞춘 라운드-하프-업(ROUND=32) 후 산술 시프트 6비트($>>6$)로 스케일을 계산하고, 동일 규칙으로 변환한 바이어스를 더한 뒤 12비트 포화(sat12)로 최종 점수를 출력한다. 이 처리 블록도 파이프라인화되어 입

력 valid와 동일한 지연으로 fc_out_data가 클래스 인덱스 0~9 순으로 스트리밍된다.

정합/검증은 (1) PyTorch QAT 경로에서 Ternary $\oplus \alpha, b, >>> 6, \text{sat}12$ 를 그대로 모사한 스크립트로 10개 로짓을 산출하고, (2) RTL 시뮬레이터 덤프와 LSB 단위로 일치하는지 비교 (3) 경계값 패턴(큰 양수/음수, 0 주변)으로 포화-라운드 동작을 점검하는 방식으로 수행했다. 결과적으로 FC는 곱셈기 제거에 따른 면적-전력 절감, 가산트리 파이프라이닝으로 인한 고클록 동작, 채널별 정규화로 인한 정확도 보전을 동시에 달성하였다.



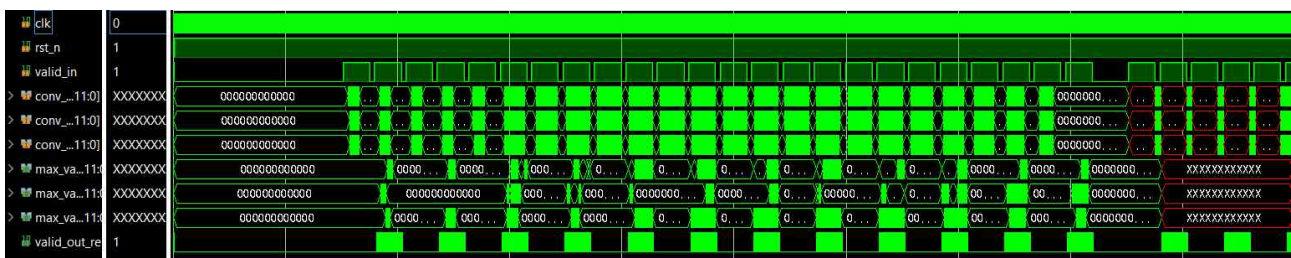
[그림 17] fc simulation 파형

3-4. maxpooling & ReLU

본 설계에서는 두 번의 2×2 MaxPooling + ReLU를 사용한다. 각 풀링 블록은 입력 특징맵(12-bit signed)을 2×2 윈도우로 스캔해 최댓값만 선택하고, 이후 ReLU로 음수는 0으로 클램프한다.

- pool1: conv1($24 \times 24 \times 3$) $\rightarrow$ $12 \times 12 \times 3$ 로 다운샘플, 잡음 억제와 지역적 불변성 강화
- pool2: conv2($8 \times 8 \times 3$) $\rightarrow$ $4 \times 4 \times 3$ 로 다운샘플, FC 입력(총 48값)로 연결

RTL 구현은 한 사이클에 한 픽셀을 처리하는 유효(valid) 스트림 기반 FSM으로, 8-valid $\rightarrow$ 4-idle 패턴을 적용해 윈도우 경계마다 비교 레지스터를 갱신한다. 내부 정수 폭은 입력과 동일한 12-bit를 유지하여 연산 오버헤드를 최소화하고 레지스터 삽입으로 valid_in $\leftrightarrow$ valid_out 지연을 상위 파이프라인과 정렬했다. 결과적으로 파라미터가 없는 연산으로 연산량과 메모리 접근을 줄이면서 후단 분류기의 견고성을 높인다.

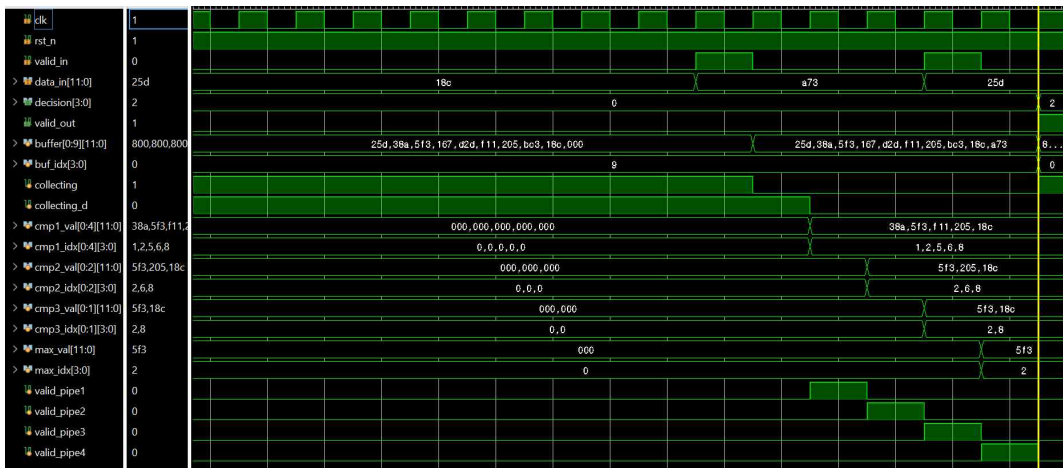


[그림 18] maxpool_relu simulation 파형

3-5. comparator

CNN 연산 마지막에 fully connected에서 받은 10개의 수 중 최댓값을 골라 최종 decision을 출력하는 모듈 또한 초기 설계에서는 파이프라이닝이 명확하게 적용되어 있지 않아 파이프라이닝을 적용하여 개선하였다. 초기 설계의 비교기는 10개의 입력을 순차적으로 비교하는 선형적인 구조로, 이는 긴 조합 논리 경로를 유발하여 전체 시스템의 타이밍 성능에 좋지 않게 작용했다.

이 문제를 해결하기 위해, 토너먼트 방식의 파이프라인을 적용하여 구조를 개선했다. 기존의 선형 비교 방식과 동일하게 총 9개의 비교기를 사용하지만, 이를 4개의 파이프라인 스테이지로 나누어 배치하였다. 10개의 입력은 10개 → 5개 → 3개 → 2개 → 1개 의 과정으로 점진적으로 비교되어 최종 최댓값이 선택된다. 이 방식을 통해 긴 경로를 단축시켜 타이밍 개선, 최대 동작 주파수 개선, 전력 효율 개선을 기대하였다. 하지만 이 또한 레지스터가 추가 됨으로 인해 면적이 증가하는 트레이드오프가 발생하지만 명확한 이점을 위하여 감수하였다.



[그림 19] 비교기 파이프라이닝 파형

4. 시뮬레이션

```
Yosys 0.55 (git sha1 60f126cd0, g++ 11.4.0-1ubuntu1~22.04 -fPIC -O3)
Time spent: 34% 2x abc (94 sec), 14% 54x opt_clean (40 sec), ...
Elapsed time: 4:35.76[h:mmin:sec. CPU time: user 269.45 sys 6.25 (99%). Peak memory: 2016352KB.
mkdir -p ./results/asap7/chip/base ./logs/asap7/chip/base ./reports/asap7/chip/base
cp ./results/asap7/chip/base/1_2_yosys.v ./results/asap7/chip/base/1_synth.v
(env time -f 'Elapsed time: %E[h:mmin:sec. CPU time: user %U sys %S (%P). Peak memory: %MKB.' /home/dayesong/OpenROAD-flow-scripts/tools/install/OpenROAD/bin
openroad -exit -no_init -threads 12 /home/dayesong/OpenROAD-flow-scripts/flow/scripts/synth_metrics.tcl) 2>&1 | tee /home/dayesong/OpenROAD-flow-scripts/flow
/logs/asap7/chip/base/1_2_yosys_metrics.log
OpenROAD v2.0-23270-g082de8efc0
Features included (+) or not (-): +GPU +GUI +Python
This program is licensed under the BSD-3 license. See the LICENSE file for details.
Components of this program may be licensed under more restrictive licenses which must be honored.
[INFO ORD-0030] Using 12 thread(s).
source /home/dayesong/OpenROAD-flow-scripts/flow/platforms/asap7/liberty_suppressions.tcl
read_liberty /home/dayesong/OpenROAD-flow-scripts/flow/platforms/asap7/lib/NLDM/asap7sc7p5t_A0_RVT_FF_nldm_211120.lib.gz
read_liberty /home/dayesong/OpenROAD-flow-scripts/flow/platforms/asap7/lib/NLDM/asap7sc7p5t_INVBUF_RVT_FF_nldm_220122.lib.gz
read_liberty /home/dayesong/OpenROAD-flow-scripts/flow/platforms/asap7/lib/NLDM/asap7sc7p5t_OA_RVT_FF_nldm_211120.lib.gz
read_liberty /home/dayesong/OpenROAD-flow-scripts/flow/platforms/asap7/lib/NLDM/asap7sc7p5t_SIMPLE_RVT_FF_nldm_211120.lib.gz
read_liberty /home/dayesong/OpenROAD-flow-scripts/flow/platforms/asap7/lib/NLDM/asap7sc7p5t_SEQ_RVT_FF_nldm_220123.lib
[INFO ODB-0227] LEF file: /home/dayesong/OpenROAD-flow-scripts/flow/platforms/asap7/lef/asap7_tech_1x_201209.lef, created 24 layers, 9 vias
[INFO ODB-0227] LEF file: /home/dayesong/OpenROAD-flow-scripts/flow/platforms/asap7/lef/asap7sc7p5t_28_R_1x_220121a.lef, created 212 library cells
Report metrics stage 1, Post synthesis...

=====
Post synthesis report design_area
=====
Design area 28968 u^2 100% utilization.
Elapsed time: 2:04.67[h:mmin:sec. CPU time: user 123.19 sys 38.27 (129%). Peak memory: 1207776KB.
/home/dayesong/OpenROAD-flow-scripts/flow/scripts/flow.sh 2_1_floorplan floorplan
Running floorplan.tcl, stage 2_1_floorplan
source /home/dayesong/OpenROAD-flow-scripts/flow/platforms/asap7/liberty_suppressions.tcl
read_liberty /home/dayesong/OpenROAD-flow-scripts/flow/platforms/asap7/lib/NLDM/asap7sc7p5t_A0_RVT_FF_nldm_211120.lib.gz
read_liberty /home/dayesong/OpenROAD-flow-scripts/flow/platforms/asap7/lib/NLDM/asap7sc7p5t_INVBUF_RVT_FF_nldm_220122.lib.gz
read_liberty /home/dayesong/OpenROAD-flow-scripts/flow/platforms/asap7/lib/NLDM/asap7sc7p5t_OA_RVT_FF_nldm_211120.lib.gz
read_liberty /home/dayesong/OpenROAD-flow-scripts/flow/platforms/asap7/lib/NLDM/asap7sc7p5t_SIMPLE_RVT_FF_nldm_211120.lib.gz
read_liberty /home/dayesong/OpenROAD-flow-scripts/flow/platforms/asap7/lib/NLDM/asap7sc7p5t_SEQ_RVT_FF_nldm_220123.lib
[INFO ODB-0227] LEF file: /home/dayesong/OpenROAD-flow-scripts/flow/platforms/asap7/lef/asap7_tech_1x_201209.lef, created 24 layers, 9 vias
[INFO ODB-0227] LEF file: /home/dayesong/OpenROAD-flow-scripts/flow/platforms/asap7/lef/asap7sc7p5t_28_R_1x_220121a.lef, created 212 library cells
=====
```

[그림 20] 합성 과정

4. 결과 및 기대효과

1. 결과

1) 정확도(accuracy)

```
----- Final Accuracy for 1000 Input Image -----
Accuracy : 96%
$stop called at time : 13381215 ps : File "C:/Users/user/vivado/CNN_1/Reference code/cnn_verilog/verilog/top_tb.v" Line 391
run: Time (s): cpu = 00:00:03 ; elapsed = 00:00:06 , Memory (MB): peak = 2500.238 ; gain = 0.000

----- Final Accuracy for 1000 Input Image -----
Accuracy : 93%
$stop called at time : 10007845 ps : File "C:/Users/dayes/verilog/CNN_AT/Adder Tree code/top_tb.v" Line 450
run: Time (s): cpu = 00:00:17 ; elapsed = 00:01:24 , Memory (MB): peak = 2667.324 ; gain = 0.000
```

(위) reference code (아래) proposed code

테스트셋 1,000장에 대해 고정소수점(Q1.15 / Q2.6)과 RTL 연산 규칙을 그대로 반영한 추론 평가 결과, 최종 정확도는 약 93%로 측정되었다. Conv1은 INT8로 유지하고, Conv2/FC에 ternary(+α)를 적용했음에도 분류 성능이 안정적으로 확보되었으며, 파이썬과 RTL 경로 간의 연산 일치가 주된 성능 유지 요인으로 확인된다.

2) 면적(area)

```
= chip ==
Number of wires:      266120
Number of wire bits:  272482
Number of public wires: 223
Number of public wire bits: 6585
Number of ports:      21
Number of port bits:  6383
Number of memories:    0
Number of memory bits: 0
Number of processes:   0
Number of cells:       225313

63
64
65
66
67
Chip area for module '\chip': 23033.877660
of which used for sequential elements: 1773.511200 (7.70%)

=== chip ===
Number of wires:      268897
Number of wire bits:  275404
Number of public wires: 9755
Number of public wire bits: 16262
Number of ports:      29
Number of port bits:  6536
Number of memories:    0
Number of memory bits: 0
Number of processes:   0
Number of cells:       253571

Chip area for module '\chip': 28967.616900
of which used for sequential elements: 7816.571280 (26.98%) ←
```

(위) reference code (아래) proposed code

합성 보고서 비교 결과, 전체 칩은 소폭 감소하였다. Conv2/FC 구간에서 곱셈기가 제거되고, 가중치가 {-1,0,+1}로 제한되면서 MUX(+/- 선택) + 가산트리 중심의 조합 논리로 단순화된 영향이 크다. 결과적으로 레지스터·BRAM 등 순차 소자는 거의 동일(파이프라인 추가분 제외) 조합 논리는 곱셈기 제거 효과로 감소, adder-tree가 지배적인 리소스가 되었으나, 파이프라이닝으로 폭발을 억제되어 총면적이 기준 대비 감소 추세를 보였다.

3) 타이밍(worst slack)

```
=====
Post synthesis report_tns
-----
tns max -1001040.81

=====
Post synthesis report_wns
-----
wns max -1618.61

=====
Post synthesis report_worst_slack
-----
worst slack max -1618.61
```

```
=====
Post synthesis report_tns
-----
tns max -1660781.88

=====
Post synthesis report_wns
-----
wns max -537.89

=====
Post synthesis report_worst_slack
-----
worst slack max -537.89
```

(좌) reference code (우) proposed code

worst-slack 보고서를 기준으로, adder-tree 파이프라이닝 이후 임계 경로는 곱셈부에서 가산트리 단계로 이동했다. ternary 적용으로 단일 사이클 콤비네이션얼 깊이는 알아졌고, 단계별 레지스터 삽입으로 데이터 경로 균형이 개선되면서 슬랙 악화 요인이 완화되었다. 다만 일부 경로에서 잔여 음수 슬랙이 관측되어, 실제 주파수 목표를 보수적으로 잡을 경우 다음 보완이 유효하다: (i) 마지막 합산 단계에 레지스터 1단 추가, (ii) 합성 시 retiming 허용, (iii) adder-tree 노드 재배선/균등 분할, (iv) conv2→pool2 경계의 fan-out 완화. 이들 조치로 P&R 이후의 WNS 여유 확보가 기대된다.

4) 전력(power)

Post synthesis report_power					
Group	Internal Power	Switching Power	Leakage Power	Total Power (Watts)	
Sequential	2.57e-02	5.77e-04	8.61e-07	2.63e-02	3.3%
Combinational	3.13e-01	4.50e-01	1.72e-05	7.63e-01	96.7%
Clock	0.00e+00	0.00e+00	0.00e+00	0.00e+00	0.0%
Macro	0.00e+00	0.00e+00	0.00e+00	0.00e+00	0.0%
Pad	0.00e+00	0.00e+00	0.00e+00	0.00e+00	0.0%
Total	3.38e-01	4.51e-01	1.80e-05	7.89e-01	100.0%
	42.9%	57.1%	0.0%		

Post synthesis report_power				
Group	Internal Power	Switching Power	Leakage Power	Total Power (Watts)
Sequential	7.61e-02	1.60e-03	3.03e-06	7.77e-02
57.6%				
Combinational	2.34e-02	3.38e-02	2.09e-05	
5.72e-02 42.4%				
Clock	0.00e+00	0.00e+00	0.00e+00	0.00e+00
0.0%				
Macro	0.00e+00	0.00e+00	0.00e+00	0.00e+00
0.0%				
Pad	0.00e+00	0.00e+00	0.00e+00	0.00e+00
0.0%				
Total	9.95e-02	3.54e-02	2.39e-05	1.35e-01
100.0%				
	73.7%	26.3%	0.0%	

(좌) reference code (우) proposed code

전력 항목 비교에서는 조합 전력의 감소와 순차 전력의 상대적 비중 증가가 동시에 나타났다. 이는 곱셈기 제거(조합 스위칭 감소)와 파이프라인 플립플롭 증가(클록·순차 전력 증가)가 상쇄적으로 작용한 결과다. 총전력은 소폭 감소하는 경향을 보였으며, 추가 최적화로 (a) 입력/가중치가 0인 탭의 operand gating/zero-skip, (b) 유효-신호 기반의 clock-gating(pool/FC 경계), (c) α -bias 레지스터의 공유/겸용 클럭 도메인 정리 등을 적용하면 더 낮출 수 있다.

5) 결론

최적화한 코드는 기존 구조 대비 1,000장 실험에서 정확도가 96% → 93%로 하락했으나, post-synthesis Worst Slack이 -1618.61 ps → -537.89 ps로 개선되어(절댓값 기준 약 66.8% 감소) 고클럭 동작 가능성이 유의미하게 높아졌다. 총 전력은 0.135 W 수준으로 뚜렷한 감소를 보였다. 반면, 단계 간 파이프라이닝 도입에 따른 레지스터 비용으로 총 셀 면적이 23033.88 → 28967.62로 약 25.7% 증가했고, 순차 소자 면적 비율도 7.70% → 26.98%로 확대되었다. 요약하면, 본 설계는 adder-tree

파이프라이닝 + ternary MAC(+ α) 적용을 통해 연산 경로를 단축하고 타이밍·전력 측면의 효율을 크게 향상시키는 대신, 정확도 소폭 저하와 면적 증가를 수용하는 명확한 트레이드오프를 보인다. 향후에는 (1) 레지스터 재배치 및 단계 균형화, (2) 마지막 합산부 컴프레서 트리 적용, (3) 합성 지시어 고정(retiming/ungroup/keep)을 통한 경로 안정화로 잔여 음수 슬랙 해소와 면적 최적화를 달성할 수 있었다.

2. 기대효과

첫 번째로, 연산 효율이 상승한다. 기존의 INT8 연산에서의 곱셈기가 가장 큰 면적과 전력을 차지한다. ternary weight를 사용하게 되면 기존의 곱셈 연산을 덧셈과 부호 반전으로 대체할 수 있기 때문에 DSP 사용량을 줄일 수 있다.

두 번째로, 기존 8bit에서 2bit로 축소되기 때문에 weight 저장 용량이 감소한다. on-chip SRAM/BRAM에 더 많은 weight를 저장할 수 있어서, DRAM access가 줄고 메모리 병목이 완화된다.

결론적으로, 곱셈기 제거와 메모리 접근 감소는 전력 소모의 두 주요 원인을 근본적으로 줄이는 효과를 가져온다. 연산부의 switching activity가 줄어들고, 외부 메모리 접근에 필요한 에너지가 감소하기 때문에 전체 시스템 차원에서 전력 효율이 향상되고 발열이 완화된다. 따라서 ternary weight를 적용하면 정확도를 크게 손상시키지 않으면서도 연산·메모리·전력 효율을 동시에 확보할 수 있는 경량 신경망 구조를 구현할 수 있다.